

Deploying a Containerized Application to a Kubernetes Cluster

Zachary Gutterman
Content Developer, Red Hat Training

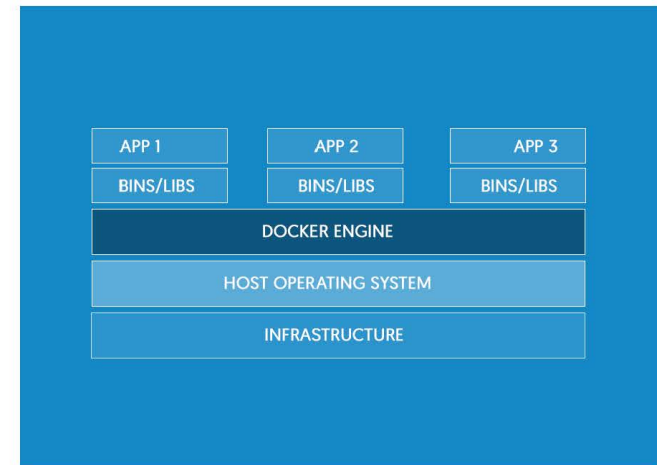
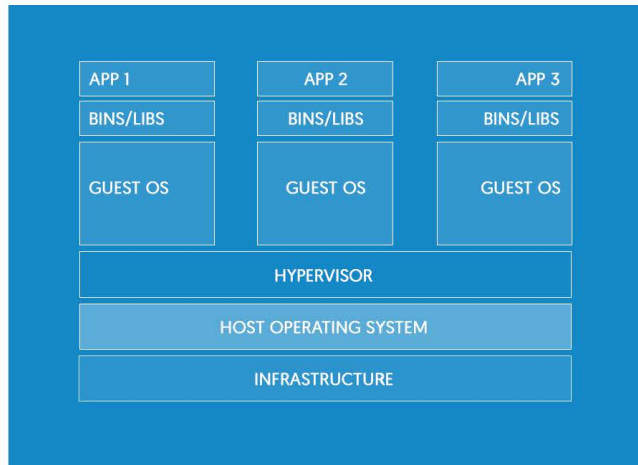
Deploying a Containerized Applications to a Kubernetes Cluster

- A brief overview of containers and Docker.
- Containerizing an application.
- Orchestrate containers with Kubernetes.
- Deploying to a Kubernetes cluster demonstration
- Synchronizing containers.

Containers

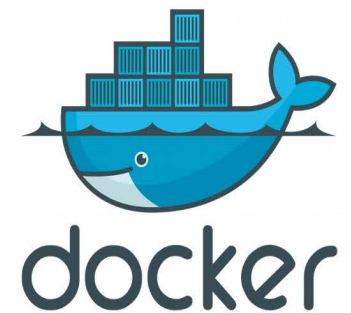
Containers

- In the old days, administrators had to manage bloated VM images, load balancers, etc. without being able to automate the process.
- Users saw a need for a fast and lightweight way to spin up VM's.
- Docker is a container implementation supported by Red Hat and RHEL Atomic Host platform.



Docker's Problem

- A single container is not long for this world. BUT, this is a feature, not a bug.
- Containers prioritize being lightweight and functional above resiliency. As a result, containers fail a lot more than individual virtual machines.
- How do we compensate for this? If containers are constantly failing, how can a cluster be highly available?



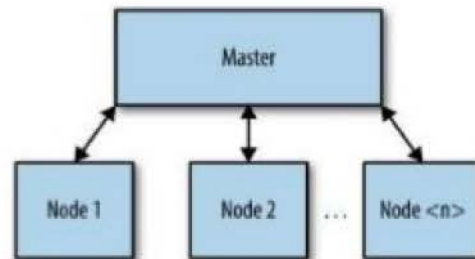
Kubernetes!

The Solution

- Containers need to be scheduled and orchestrated so that if one fails another will spin up to maintain the cluster's integrity.
- Primary Challenges:
 - Orchestration
 - Units need to be well balanced to distribute the load.
 - Scheduling
 - Failures need to be replaced near instantaneously.
 - Isolation
 - A failure of one computing unit cannot take down another.

Hello, Kubernetes

- Kubernetes is an open source container cluster manager developed by Google released in 2014.
- Used by Google to schedule and launch around 7,000 containers a second.
- Kubernetes follows the master-slave architecture.



Kubernetes Architecture

- A Kubernetes cluster is a set of servers (**nodes**) that run Docker containers.
- Nodes also run the process Kubelet whose job it is to respond from commands from the master.
- The **master** acts as the controller, managing the workload and communication in a Kubernetes cluster.
- **Pods** are a collection of containers that are bundled together because they share a common resource.
 - Note: Pods can also have just one container.

Volumes

- Container storage is, by nature, ephemeral, meaning its contents are not preserved after the container is removed.
- Because Kubernetes volumes are on the pod level instead of container level, temporary data written by a container will not be lost if that container goes down.
- If users want to persist data, they must use **Persistent Volumes (PV)**. Or, persistent storage for pods that can be mounted inside a container to store data.
- **Persistent Volume Claims (PVC)** represent a request for storage by a pod to be bounded to an available PV. This allows multiple containers to use the same PV and share its contents

Replication Controller

- **Replicas** refer to multiple copies of a pod. Replicas provide high availability to a cluster.
- The **Replication Controller** manages the replicas by creating and destroying replicas to maintain the specified equilibrium.
- A replication controller follows the rules for the cluster described in a **pod template**.

Services

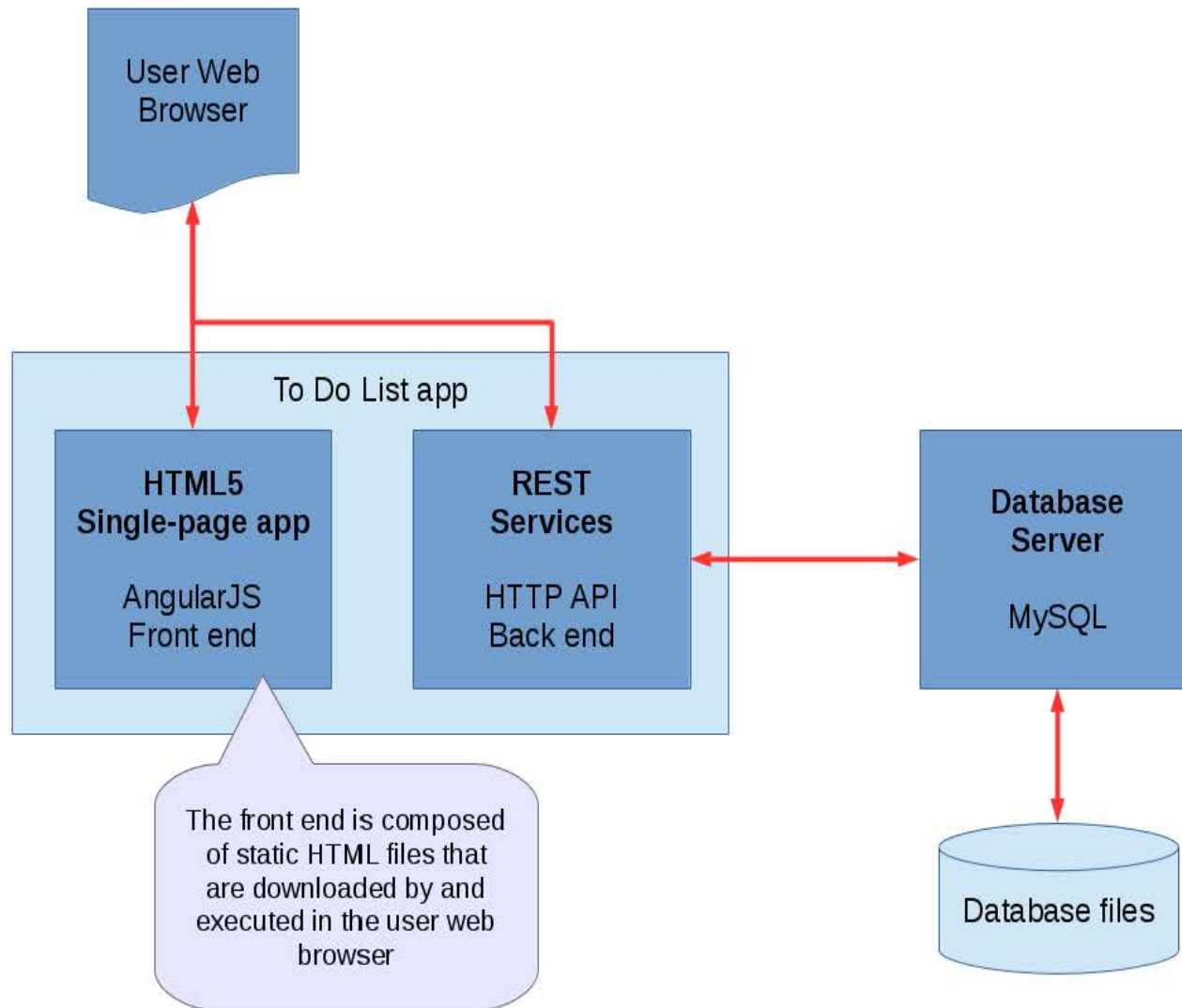
- What happens if you have a pod running a MySQL container and your front end in another container that is hardcoded to the MySQL pod's IP address and that IP address changes?
- **Services** provide persistent end points to a set of pods in a cluster.
- A service is comprised of a single IP and port combination. This solves the dilemma of keeping up with every transient IP address assigned by Docker.

Controlling Kubernetes

- **Kubectl** is the command line responsible for creating and manipulating Kubernetes resources.
- Kubernetes resources are defined in YAML files.
- The following are some of the useful commands:
 - **Create** – Creates a resource by a filename.
 - **Delete** – Deletes resource by filename.
 - **Describe** – Details information about a resource
 - **Get** – Displays one or many resources.
 - **Scale** – Changes the number of replicas for a Replication Controller

The To Do List Apppplication

The To Do List Application Architecture



Demonstration: Deploying the To Do List Application on a Kubernetes Cluster

Demonstration: Scaling the To Do Application

But be careful...

- Imagine a scenario where a Wildfly application server depended on a MySQL server before starting up. How would Docker or Kubernetes know to pause the Wildfly containers until the MySQL containers or pods are available?
- It wouldn't!

Demonstration: The wait.sh Script

Learn More

- DO276: Containerizing Software Applications (Docker and Kubernetes)
- DO290: OpenShift Enterprise Development
- DO280: OpenShift Enterprise Administration
- www.redhat.com/en/services/training